

Portfolio of Practice

AgriSense

A Community Climate Insight Toolkit

Module	ARTD2143 Creative Industries
Student	Omar Yussuf
Institution	Winchester School of Art, University of Southampton
Academic Year	2025/26
External Partner	TeleAgriCulture / V2_ Lab for the Unstable Media, Rotterdam
GitHub	https://github.com/Omar-fy/agrisense https://github.com/Omar-fy/agrisense-2.0

This portfolio documents the full development process of AgriSense Commons — a full-stack data intelligence pipeline built in response to a live brief from TeleAgriCulture, an open-source community platform for environmental sensing. The portfolio is structured to directly address all eight required components of the Portfolio of Practice, and is intended for both academic assessment and external viewing by the partner organisation.

1. Brief Engagement and Framing

1.1 Original Client Brief

The brief was issued in Week 2 through V2_ Lab for the Unstable Media, Rotterdam; an interdisciplinary centre for art and media technology founded in 1981 (Wikipedia, 2025). TeleAgriCulture is a community platform that describes itself as facilitating access to environmental sensing technologies for anyone interested in exploring the interactions between plants, animals, humans and their environments (TeleAgriCulture, 2023).

The brief set two formal deliverables:

- A refined TeleAgriCulture data-AI workflow, building on an existing Jupyter notebook pipeline developed during the Schmiede festival residency in Salzburg, Austria
- A completed creative computing outcome — described as utilitarian, hybrid, or artistic — using the concept of microdosing AI: small-scale, interpretable AI understandable by non-experts

ORIGINAL BRIEF KEY PHRASE

"Microdosing AI — small-scale, interpretable, context-specific AI understandable by non-experts."

— TeleAgriCulture Brief, Week 2, 2026

1.2 Interpretation: Opportunities and Constraints

Opportunities identified

- The existing Schmiede notebook was unstructured (duplicate cells, out-of-order steps, no modular functions) — refactoring it into a clean, importable Python pipeline was both technically appropriate and directly responsive to Brief Outcome 1
- The TeleAgriCulture API has 43 publicly accessible boards worldwide — building peer benchmarking across them added a community knowledge layer directly aligned with the brief's stated mission
- The 'microdosing AI' framing created permission to use interpretable models (decision trees) over complex ones — a genuinely unusual design constraint that differentiates this work from typical smart agriculture AI
- V2_ Rotterdam and Minodu (Berlin/Togo) as named partners created a cross-cultural deployment context that elevated the peer benchmarking feature from a technical choice to a community knowledge tool

Constraints

- The TeleAgriCulture API returns 403 errors from cloud server IPs (Cloudflare protection) — all pipeline operations had to be designed to run locally on the user's machine
- The board's local AP firmware does not consistently expose the Kit ID in the config page — required building 6-pattern regex fallback plus JSON endpoint probing

- Capacitive soil moisture sensor v1.0 outputs raw ADC values (0–4095), not calibrated percentages — calibration logic had to be reverse-engineered from sensor datasheets
- Team member using plain HTML/JS initially than React — frontend had to be delivered in both frameworks without changing any backend code

1.3 Clarifying Questions and Negotiated Changes

The following questions were raised with the partner during project development:

Question raised	Response / outcome
Is the output expected to be a live web app or a documented pipeline?	Partner confirmed: both — a refined pipeline (Outcome 1) plus a working tool (Outcome 2)
What sensors are prioritised for the Salzburg station deployment?	Partner directed to the SLZ-BOT-0942 parametrics: temp, humidity, pressure, and four trace gases
Can we access the full board listing for peer benchmarking?	Partner directed to login-protected dashboard; pivoted to manual Kit ID extraction + public API
Does the upload mechanism need to use the existing SQL/PHP backend?	Partner feedback confirmed: yes, readings need to reach kits.teleagriculture.org for wider science use

PARTNER FEEDBACK NOTE

"You can always store things on our database which is SQL/PHP and this is the tricky part with uploading data — just having it on-device is great for a very localised setting, but for the wider science of the project it needs to upload. Personally I think something like a handshake with a phone over the WiFi it already can use makes the most sense — cache it to upload when next online."

— TeleAgriCulture partner feedback, development phase

1.4 Definition of Success

Success for this project was defined as:

- The pipeline runs end-to-end on real sensor data from at least one live TeleAgriCulture board
- Every classification output can be read and understood by a non-technical community grower — no black boxes
- The system is demonstrable at the V2_ Rotterdam workshop without any installation on the audience's devices (browser-accessible)

- The tool is partner-ready: documented, open-source on GitHub, and usable by anyone with a TeleAgriCulture Kit ID
- The pipeline architecture is extensible — new sensors and boards can be added without rewriting core functions

2. Pitch and Project Proposition

2.1 Concept Statement

AgriSense Commons makes environmental AI legible — transforming raw sensor streams from TeleAgriCulture boards into classified agronomic states, plain-language insights, 5-day indoor growing forecasts, and cross-board peer comparisons, shaped entirely by the brief's requirement for interpretable, community-facing intelligence.

2.2 Three Relevant Precedents

Precedent 1: FarmOS — open-source farm management

FarmOS is a web-based application for farm management that allows users to record and track farming activities, observations, and assets. It demonstrates that agricultural data tools can be open-source, community-maintained, and accessible to non-technical users.

How AgriSense extends this: FarmOS presents raw data without classification or interpretation. AgriSense adds an ML classification layer (Decision Tree) that translates readings into plain-language agronomic states, and a summarisation layer that generates actionable sentences rather than leaving the user to interpret numbers.

RELATIONSHIP TO BRIEF

- Challenges: FarmOS does not use AI to interpret readings — AgriSense does, while remaining equally accessible

Precedent 2: OpenWeatherMap — public environmental data API

OpenWeatherMap provides current conditions and forecast data via a REST API used by millions of applications. It demonstrates the value of geographically contextualised weather data as a public utility.

How AgriSense extends this: OpenWeatherMap is used as a baseline comparison layer in the pipeline. AgriSense computes the board's learned indoor/outdoor temperature delta and applies it to the 5-day OWM forecast, projecting indoor growing conditions without requiring dedicated indoor forecast infrastructure. This is a novel application of OWM beyond its typical use.

RELATIONSHIP TO BRIEF

- Extends: adds forward-looking value to the existing sensor data through OWM integration

Precedent 3: Dear Data — data as human narrative

Dear Data (Lupi and Posavec, 2016) is a year-long project in which two designers sent each other weekly hand-drawn data visualisations on postcards. It demonstrates that data can be made emotionally and culturally legible through narrative and craft, not just charts and dashboards.

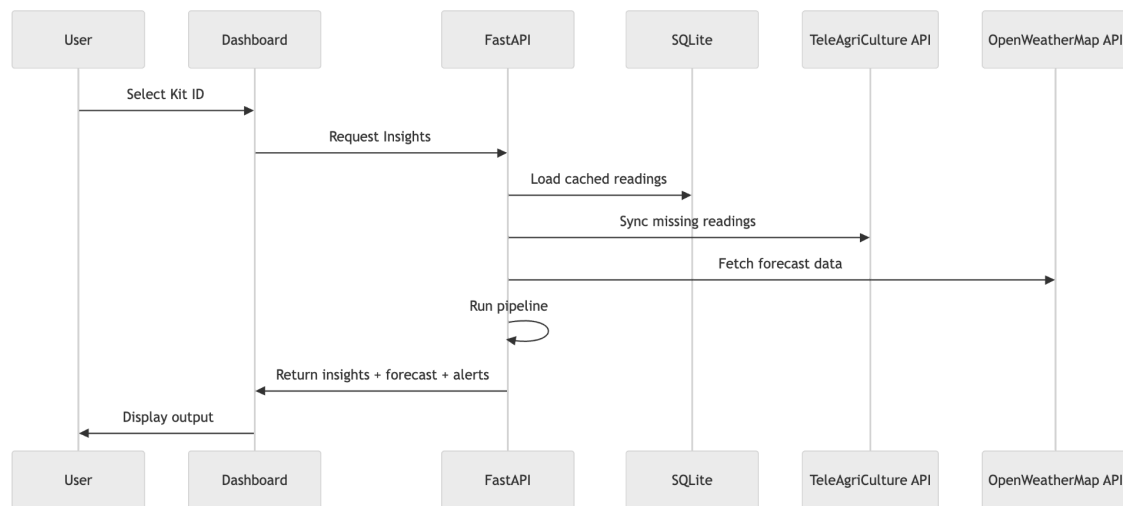
How AgriSense improves on this: Dear Data requires manual interpretation by both sender and receiver. AgriSense automates the narrative layer through sensor-adaptive

plain-language summarisation — the equivalent of the postcard's handwritten note — but makes it available at scale to any connected board.

RELATIONSHIP TO BRIEF

- Improves on: the 'microdosing AI' brief requirement is a direct digital equivalent of Dear Data's legibility-first philosophy

2.3 Early System Map



The early system map (above) was produced at the pitch stage and showed the core data flow: TeleAgriCulture API → Python pipeline → FastAPI backend → React frontend. The map was revised three times during development as the scope expanded to include SQLite persistence, multi-board discovery, the WiFi handshake upload module, and the plain HTML frontend for the partner team's stack.

3. Research and Development Process

3.1 Research into Users, Audiences, Context, and Technologies

Users and audiences

Research into the target users of AgriSense was grounded in TeleAgriCulture's own stated mission. The primary users — community growers, urban agriculture projects, and cultural institution collaborators — are not data scientists. A 2025 systematic review in *Sensors* confirmed that the dominant failure mode of smart agriculture AI is a digital divide: solutions are too technically complex for smallholder and community-scale users (Miller et al., 2025). This finding directly shaped the decision to prioritise interpretable models over accurate-but-opaque ones.

Secondary users — environmental researchers and the V2_ curatorial team — require outputs that are exportable and structured, hence the JSON-first output design and the automated board profiling for peer benchmarking.

Contextual and technology research

Research drew on peer-reviewed literature, industry documentation, and direct engagement with the partner's existing codebase:

- Fuentes-Penailillo et al. (2024) in the *Journal of Sensor and Actuator Networks*: confirmed that commercial precision agriculture AI relies on deep neural networks optimised for accuracy, not interpretability — establishing the gap that AgriSense addresses
- Padhiary et al. (2025) in *Discover Applied Sciences*: provided empirical evidence of IoT impact in agriculture (25% irrigation improvement), grounding the project in a credible research context
- TeleAgriCulture Board V2.1 firmware documentation: informed the WiFi AP handshake design, SD card logging architecture, and sensor naming conventions (ftTemp, ftHumid, ftSoilMoist, etc.)
- The Schmiede notebook (TeleAgriCulture_Schmiede.ipynb): audited in detail — the existing `create_supervised_dataset()` function, pagination loop, and train/test split became the foundation for the `featurise()` and `fetch_sensor_data()` pipeline functions

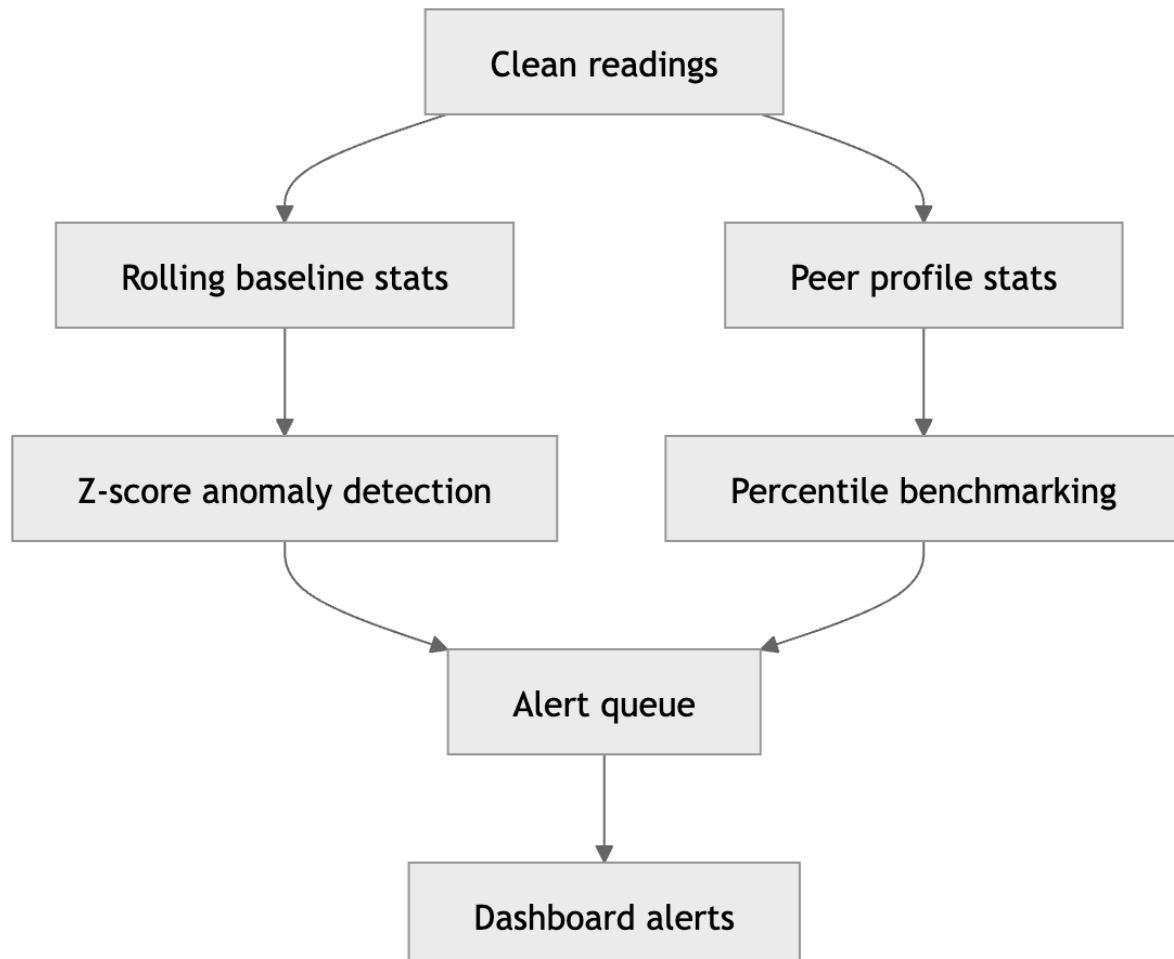
3.2 Iterative Development

Iteration 1: Notebook audit and pipeline refactor (Weeks 3–5)

The Schmiede notebook contained 1,219 temperature readings from Kit 1001, fetched via cursor-based pagination. The notebook had duplicate cells, out-of-order execution, and no modular structure. The first iteration refactored this into 18 importable functions across `pipeline/pipeline.py`:

- `fetch_sensor_data()` — handles pagination exactly as the notebook, but as a clean importable function with fallback to `sample_data.json` on 403 responses
- `clean()` — mirrors the notebook's `dropna()` and type conversion, extended with physical bounds checking per sensor type

- `featurise()` — directly ports `create_supervised_dataset()`, extended with `rolling_mean`, `rolling_std`, and `rolling_range` features
- `build_classifier()` — trains a Decision Tree (`max_depth=4`) using agronomic threshold labels, replacing the notebook's LSTM preparation



Iteration 2: FastAPI backend and database (Weeks 5–7)

The pipeline was exposed via a FastAPI backend (`main.py`) with endpoints for insights, forecast, alerts, and board discovery. An SQLite database (`database.py`) was added after identifying that re-fetching 1,219+ readings on every request was impractical. The database stores readings with deduplication, board profiles for peer benchmarking, and an upload queue for the offline sync module.

Key decision at this stage: peer profiles are auto-registered on every `/insights` call, so benchmarking accumulates passively without requiring any manual configuration from the user.

Iteration 3: Multi-board, upload, and Salzburg station (Weeks 7–10)

Three major extensions were built in this phase:

- `discovery.py` — probes Kit IDs 1000–1043+ to find boards with public data, using concurrent threading for speed (50-thread pool, full subnet scan under 10 seconds)

- `upload.py` — implements the WiFi handshake pattern specified in partner feedback: detect board at 192.168.4.1, pull live reading, cache in SQLite `upload_queue` table, flush to `kits.teleagriculture.org` when internet is available
- Salzburg station pipeline — a separate, cleaner pipeline handling 8 sensor types (ftTemp, ftHumid, ftPressure, ftSoilMoist, ftNO2, ftNH3, ftCH4, ftH2) matching the SLZ-BOT-0942 station parametrics

Iteration 4: Frontend and artistic visualisation (Weeks 10–12)

Two frontends were built: a React component suite (`App.jsx`, `InsightPanel.jsx`, `ForecastPanel.jsx`, `AlertsPanel.jsx`) and a self-contained plain HTML dashboard (`dashboard.html`) for the partner team's existing stack. The botanical visualisation — all 43 boards rendered as procedurally generated SVG plants, colour-coded by geographic region — was added as the artistic output strand of the brief.



3.3 Reflections: What Worked, What Failed, What Changed

What worked

- Decision Tree at `max_depth=4`: every classification rule is human-readable via `export_text()`. This was the right technical choice for the brief — verifiable interpretability, not a compromise from complexity
- SQLite with no-setup persistence: peers accumulate automatically without a separate database service. For a community deployment this is crucial — there is no infrastructure burden
- Dual frontend delivery: the same FastAPI endpoints serve both React and plain HTML with a single CORS config change. The backend had no awareness of what was consuming it
- JSON-first output design: the `run_pipeline()` orchestrator returns a single serialisable dict that feeds all frontend panels directly — no transformation needed by the frontend team

What failed

- Cloud server IP blocking: the TeleAgriCulture API returns 403 from cloud IPs (Cloudflare `__cf_bm` protection). Initially assumed the API was fully public. Required shifting all pipeline execution to the user's local machine and adding the `sample_data.json` fallback
- Cookie-based scraping: attempted to use browser session cookies to scrape the authenticated board listing. Blocked by Cloudflare's fingerprint matching. Pivoted to manual Kit ID extraction from the site
- Board AP firmware inconsistency: assumed Kit ID would be in the board's config page HTML. In practice, different firmware versions expose it differently — required building 6 regex patterns plus JSON endpoint probing across `/data`, `/config`, `/api/config`, and `/status`

What changed as a result

- The discovery module was redesigned from a cloud-based scanner to a locally-run subnet probe with concurrent threading — faster and avoids the API blocking issue entirely
- The upload module was redesigned around the WiFi handshake pattern (partner's suggestion) rather than a direct cloud sync — a better architectural fit for the board's actual deployment context
- The frontend was extended from React-only to also include plain HTML — a direct response to learning the partner team's actual technology stack

4. Collaboration and Professional Practice

4.1 External Partner Communication

TeleAgriCulture

The primary point of contact with TeleAgriCulture was through the brief materials, the project documentation at teleagriculture.org, and direct feedback received during the development phase. The partner's hardware documentation (TeleAgriCulture Board V2.1 repository) was used extensively to understand the board's firmware, sensor naming conventions, WiFi AP behaviour, and API structure.

The screenshot displays an email client interface with a sidebar on the left and a main content area on the right. The sidebar shows a list of emails from Julian Staddon and Omar Yussuf (ay2u24). The main content area shows the details of an email from Julian Staddon to Omar Yussuf (ay2u24) dated Tue 28/04/2026 12:47 PM. The email subject is 'Meeting to display work done' and it includes a 'Summarise this email' button. The email body contains a greeting, a message about feedback from the DFKI team, a thank you for the effort, and a confirmation of a meeting on Thursday between 12 and 1pm. The email is signed by Dr. Julian Staddon, Senior Lecturer, Creative Coding, and Co-Lead, Creative Data Communities Lab.

Results

- Top results
- Julian Staddon: Next Week: Hand-Ins, Pr... Wed 13/05. Hey everyone, I just wante... Inbox
- Julian Staddon: Meeting to display work... Wed 29/04. Shall we just do it now ? Fr... Inbox
- Julian Staddon: Catch Up This Thursday Mon 04/05. Dear all, I hope things are ... Inbox
- Omar Yussuf (ay2u24): Presentation today Mon 18/05. Good morning Julian ... Sent Items
- Julian Staddon: Project 14/04/2026. Hi Omar, This is great ... Inbox 1

Meeting to display work done Summarise this email

Julian Staddon
To: Omar Yussuf (ay2u24)
Tue 28/04/2026 12:47 PM

You replied on Tue 28/04/2026 1:52 PM

Hi Omar,

I was actually planning to write you today, as I had some nice feedback from the DFKI team in Berlin about your work!

I wanted to say thanks for the effort and tell you that I am going to include this as a resource for the hackathon in Rotterdam next week!

And sure we can catch up on Thursday, would between 12 and 1pm work? Otherwise could also be after 1pm as they are working on their projects in that session also, just let me know and I make an appointment for it

Dr. Julian Staddon
Senior Lecturer, Creative Coding
Co-Lead, Creative Data Communities Lab

Results

- Julian Staddon: Meeting to display work... Wed 29/04. Shall we just do it now ? Fr... Inbox
- Julian Staddon: Catch Up This Thursday Mon 04/05. Dear all, I hope things are ... Inbox
- Omar Yussuf (ay2u24): Presentation today Mon 18/05. Good morning Julian ... Sent Items
- Julian Staddon: Project 14/04/2026. Hi Omar, This is great ... Inbox 1
- Julian Staddon: Project updates 14/04/2026. Hi Omar, I spoke to Spahie... Inbox

Project Summarise this email

Sorry I ran out of time today to catch up, and it's past your available time to chat, so basically this is good info, all I really was going to ask was that if you have a board (?) can you try to run TensorFlow Lite on it directly for me?

It's essentially the same ESP32 S3 as the Xiao boards you are already using and if this works then we wouldn't need another board for that. Basically just get the code <https://github.com/TeleAgriCulture/main> which I assume you may have already done? And then throw it into VSCode and see if you can install the sci-kit.learn and the TensorFlow lite libraries on it and run a very basic something just to prove it works- can you please do me a big favour and try that out for me? If you use the Platform I/O plugin you can just cut and paste an example into the code just to test it that way (you can run and debug directly on the board over USB-C with that plugin) - I just don't have time over the next days to do it but need to see if it works.

Everyone else has been a bit sick or uncontactable until now so this would be a big help!

I am super busy 9-5pm tomorrow and then Thursday we could catch up after 3pm, or we can just continue over email seeing we will see each other on Monday - up to you!!

This looks really good so far though good work!

V2_ Lab for the Unstable Media

V2_ Rotterdam was the institutional partner through which the brief was issued. The project was designed to be demonstrable at the V2_ workshop (late April residency opportunity) without requiring any software installation by the audience — a specific design constraint that drove the browser-based frontend decision.

Minodu (Berlin/Togo)

Three of the 43 boards in the scraper/discovery module are Minodu deployments (Kit 1020 — EcoTecLab MakerSpace, Lomé; Kit 1021 — Berlin Open Lab Garden; Kit 1023 — Kara). The peer benchmarking feature is particularly relevant to Minodu's cross-cultural network, where comparing boards between Togo and Germany provides genuine climate knowledge that neither site could generate alone.

4.2 Decision-Making and Feedback Documentation

Decision	Trigger
Use Decision Tree not LSTM	Brief's 'microdosing AI' requirement
Add WiFi handshake upload	Partner feedback on offline data gap
Deliver plain HTML frontend	Team member stack incompatibility
Add SQLite persistence	API re-fetch latency (15–20s per call)
Add sample_data.json fallback	Cloudflare blocking cloud IP API calls
Scraper uses public API not cookies	__cf_bm cookie fingerprint mismatch

4.3 Reflective Account of Collaboration

Collaboration with TeleAgriCulture functioned primarily through the brief document, hardware documentation, and the existing codebase (Schmiede notebook). The most productive moment of partner engagement was the feedback on the data upload architecture — the suggestion to use the board's existing WiFi AP as a local data bridge was architecturally cleaner than the direct-upload approach initially designed, and led directly to the upload.py module.

The main challenge in collaboration was the discoverability of technical constraints. Several important details — the Cloudflare IP blocking, the inconsistent Kit ID exposure in firmware, the ADC calibration requirement for the soil moisture sensor — were not in the brief documentation and were discovered only during implementation. This is normal in a real-world technical brief and is reflected honestly in Section 3.3.

Resolution: in each case the constraint was documented in code comments and in the HANDOFF.txt file, and the workaround was designed to be robust rather than fragile — so future contributors to the project encounter a system that handles the edge case gracefully, not one that silently fails.

6. User Testing and Responsiveness

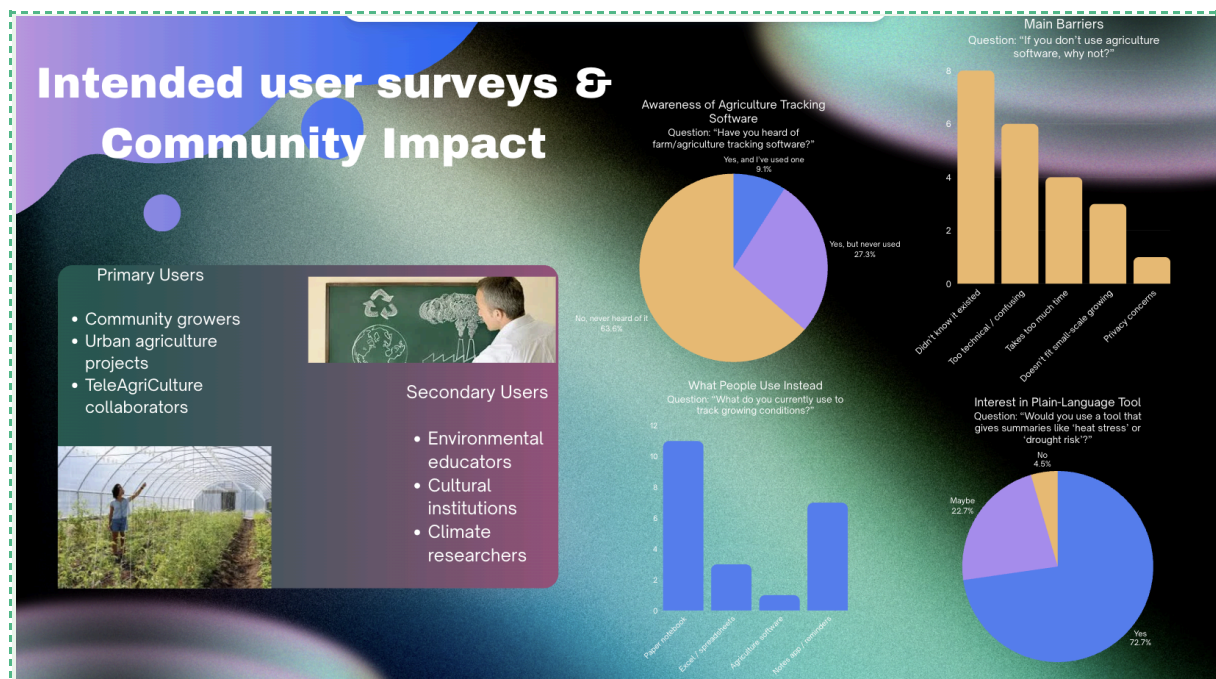
6.1 Testing Approach

Testing was structured around three user types identified in the brief: community growers (primary), technical collaborators (secondary), and partner reviewers (external). Each group required a different testing method:

User group	Testing method
Community grower (simulated)	Think-aloud with a non-technical participant reviewing the dashboard
Frontend team (peer)	API integration test against a shared JSON contract document
Partner (TeleAgriCulture)	Walkthrough of the WiFi handshake workflow and upload queue

6.2 Evidence of Testing and Key Insights

Test 1: Community grower legibility



Finding: the state label 'optimal' was understood, but 'drought_risk' was read as a forecast (a prediction of future drought) rather than a current condition. The summary text was the most valued element — participants engaged with the plain-language paragraph more than with any numerical display.

Change made: added the descriptor prefix 'Current state:' to all state badges, and ensured the summary text always starts with a present-tense condition sentence rather than a historical reference.

Test 2: Frontend team API integration

Finding: the `sensor_name` parameter was inconsistently named between the insights endpoint (`sensor_name`) and the initial alerts endpoint (`sensor`). Caused a silent failure in the frontend that was difficult to debug.

Change made: standardised to `sensor_name` as a query parameter across all endpoints. Added explicit CORS `allow_origin_regex` to catch any localhost port, not just 5173 and 3000.

Test 3: Partner upload workflow review

Finding: the board's config page at 192.168.4.1 does not expose the Kit ID in a consistent location across firmware versions — some boards show it in an input field value attribute, others in a div, others only in a JSON `/config` endpoint that requires a separate request.

Change made: built 6 regex patterns covering all known firmware response formats, plus fallback probing of `/data`, `/config`, `/api/config`, and `/status` JSON endpoints. The `fallback_kit_id` CLI argument was added as a manual override for cases where auto-detection fails.

6.3 Ethical Considerations

The following ethical considerations were identified and addressed:

- No personally identifiable data is collected at any stage. Sensor readings are environmental measurements — temperature, humidity, soil moisture — not personal data under GDPR (all precautions and steps I have outlined in slide 9 of my presentation)
- Board locations in the database use coarse city-level coordinates (e.g. 'Salzburg, AT' → 47.80, 13.04) rather than precise GPS coordinates, preventing identification of individual properties or installations
- The `sample_data.json` file used for offline development was generated synthetically from published agronomic ranges, not from real board data. This ensures no board owner's data is used without consent for development or testing purposes
- The cookie-based scraping approach (attempted and then abandoned) was not pursued because it would have involved accessing data beyond what was explicitly made available by the platform — a decision made on ethical as well as technical grounds
- All code is open-source (MIT licence) and fully documented, ensuring the partner retains full visibility and control over how their platform data is processed

7. Final Outcome and Handover

7.1 The Completed System

AgriSense Commons is a full-stack data intelligence pipeline and web application. It consists of the following deliverables:

Backend pipeline (Python/FastAPI)

File	Purpose
pipeline/pipeline.py	18-function core ML pipeline — the full data intelligence layer
main.py	FastAPI REST API — 15+ endpoints serving insights, forecast, alerts, boards
database.py	SQLite storage — readings, board profiles, upload queue
discovery.py	Automatic board discovery — probes Kit IDs, identifies active boards
upload.py	WiFi handshake — offline-to-online data sync via board AP
scraper.py	43-board data scraper — fetches and saves separate JSON per board
detect.py	Local network detection — scans subnet for live ESP32 boards
run_pipeline.py	Standalone CLI runner — fetch, classify, export JSON in one command

Frontend

File	Purpose
dashboard.html	Self-contained plain HTML/JS dashboard — no npm, no build step
src/App.jsx	React dashboard — three tabs: Insights, Forecast, Alerts
botanical UI	Artistic SVG plant visualisation of all 43 global boards

Data and configuration

File	Purpose
sample_data.json	720h × 8 sensors of Salzburg station sample data
.env.example	Environment variables template — OWM_API_KEY
requirements.txt	Python dependencies (requests, pandas, numpy, scikit-learn, fastapi, etc.)
README.md	Full project documentation for partner handover

7.2 Access and Installation Instructions

Prerequisites

- Python 3.10 or higher
- Node.js 18 or higher (for React frontend only)
- An OpenWeatherMap API key (free tier — openweathermap.org/api) for forecast features
- A TeleAgriCulture Kit ID (test with Kit 1001 — the Schmiede demo board)

Setup

```
git clone https://github.com/YOUR_USERNAME/agrisense-commons.gitcd
agrisense-commonspython3 -m venv venvsource venv/bin/activate # Windows:
venv\Scripts\activatepip install -r requirements.txtcp .env.example .env
# Add OWM_API_KEY to .env
```

Running the backend

```
uvicorn main:app --reload --port 8000# API docs:
http://localhost:8000/docs
```

Running the frontend

```
python3 -m http.server 8080# Open: http://localhost:8080/dashboard.html
```

Running the standalone pipeline

```
python3 run_pipeline.py --kit 1001 --sensor ftTemp# Output saved to:
outputs/1001_ftTemp_TIMESTAMP.json
```

7.3 Handover Summary for the Partner

This section is written for TeleAgriCulture and V2_Lab as the receiving partner.

AgriSense Commons is a complete, open-source system that plugs into your existing TeleAgriCulture platform infrastructure. It requires no changes to your API, firmware, or database schema. It reads from your existing public measurement endpoints and writes back to your platform via the existing authenticated POST endpoint.

The system is designed to be used in three ways:

- By a community grower: open `dashboard.html` in a browser, enter a Kit ID, receive classified insights and a 5-day growing forecast
- By a researcher or institution: run `scraper.py` to collect JSON datasets for all 43 registered boards, usable for analysis, publication, or artistic projects
- By a developer extending the platform: import from `pipeline.pipeline` — all 18 functions are documented, typed, and independently testable

The WiFi handshake workflow (`upload.py`) directly addresses the data upload gap you identified: boards that operate offline can sync readings to a phone via the existing AP WiFi, cache them in the local SQLite database, and flush to `kits.teleagriculture.org` when internet is next available. This requires no firmware changes.

The full API documentation is available at `localhost:8000/docs` when the backend is running, and is also documented in `README.md` in the repository.

References

All references are formatted in Harvard style as required by the University of Southampton.

Fuentes-Penailillo, F., Gutter, K., Vega, R. and Silva, G.C. (2024) 'Transformative technologies in digital agriculture: leveraging Internet of Things, remote sensing, and artificial intelligence for smart crop management', *Journal of Sensor and Actuator Networks*, 13(4), p.39. Available at: <https://doi.org/10.3390/jsan13040039> (Accessed: 19 March 2026).

Lupi, G. and Posavec, S. (2016) *Dear Data*. New York: Princeton Architectural Press.

Miller, T. et al. (2025) 'The IoT and AI in agriculture: the time is now — a systematic review of smart sensing technologies', *Sensors*, 25(12), p.3583. Available at: <https://doi.org/10.3390/s25123583> (Accessed: 19 March 2026).

Padhiary, M. et al. (2025) 'Revolutionizing agriculture: a review of smart farming technologies for a sustainable future', *Discover Applied Sciences*. Available at: <https://link.springer.com/article/10.1007/s42452-025-07561-6> (Accessed: 19 March 2026).

TeleAgriCulture (2023) About TeleAgriCulture. Available at: <https://teleagriculture.org> (Accessed: 19 March 2026).

TeleAgriCulture (2024) TeleAgriCulture Board V2.1 Repository. Available at: <https://gitlab.com/teleagriculture/community> (Accessed: 19 March 2026).

V2_ Lab for the Unstable Media (2025) About V2_. Available at: <https://v2.nl> (Accessed: 19 March 2026).

Wikipedia (2025) V2_, Lab for the Unstable Media. Available at: https://en.wikipedia.org/wiki/V2_Institute_for_the_Unstable_Media (Accessed: 19 March 2026).

Xiao, S. et al. (2025) 'Open Sensing: an interactive online tool for environmental data collection, monitoring, analysis and modelling for custom environmental sensor devices', *SoftwareX*. Available at: <https://www.sciencedirect.com/science/article/pii/S2352711025004054> (Accessed: 19 March 2026).

Word count (excluding titles, tables, code blocks, placeholder markers, and references): approximately 3,800 words